

Inference Engineering – Week Assignment

Part A: Theory

Question 1: Latency vs. Throughput

Latency is the time taken to complete a *single* inference request – e.g. the wall-clock time from sending a prompt to receiving the full response (or, for generative models, the time-to-first-token and the time between subsequent tokens). It is measured in units of time (ms/s) per request.

Throughput is the number of requests (or tokens) a system can process per unit time across *many* concurrent users, e.g. requests/sec or tokens/sec. It is a system-level, aggregate measure rather than a per-user measure.

Which one matters more, and why? It depends entirely on the deployment scenario, and the two are usually in tension with each other:

- **Interactive / user-facing applications** (chatbots, copilots, voice assistants) are latency-bound. A user waiting for a response cares about how fast *their* answer comes back, not how many other users are being served. Here, low latency (especially time-to-first-token) dominates perceived quality.
- **Offline / batch workloads** (bulk document summarization, embedding generation, dataset labeling) are throughput-bound. No one is waiting synchronously, so the goal is to maximize total tokens processed per second per GPU/CPU to minimize cost.

The tension arises because the main lever used to raise throughput – **batching** multiple requests together so the accelerator processes them in parallel – increases the latency of any individual request in the batch, since a request may have to wait in a queue for the batch to fill, and larger matrix multiplications take marginally longer. Conversely, running requests one at a time (batch size 1) minimizes latency but wastes compute (the hardware is often memory-bandwidth bound, not compute bound, for a single request), giving poor throughput. Real serving systems (e.g. dynamic / continuous batching) try to find a middle ground: batch as much as possible without violating a target per-request latency Service Level Objective (SLO).

Question 2: KV Cache

Transformer decoders generate text autoregressively – one token at a time, where each new token’s attention computation depends on *all* previous tokens in the sequence. For every layer, self-attention requires the Key (K) and Value (V) projections of every previous token to compute the attention output for the current token:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V$$

Without caching, generating token t would require re-feeding the entire sequence $[1, \dots, t]$ through the model and recomputing K and V for *every* past token from scratch – an $O(t)$ amount of redundant work repeated at every one of the n generation steps, giving $O(n^2)$ total work for a sequence of length n .

The **KV cache** avoids this by storing the computed K and V tensors for every past token (per layer, per attention head) in memory. At each new generation step, the model only needs to:

1. Compute Q , K , V for the *single new token*.
2. Append the new K , V vectors to the cache.
3. Attend the new Q over the *entire cached* K , V (old + new).

This reduces the per-step cost from $O(t)$ recomputation to $O(1)$ new computation plus a cheap read of the cache, turning generation into an $O(n)$ process overall instead of $O(n^2)$. The trade-off is memory: the KV cache grows linearly with sequence length, batch size, number of layers, and number of heads, and is often the dominant consumer of GPU/CPU memory during long-context generation – which is exactly the problem optimizations like PagedAttention (Question 4) address.

Question 3: Quantization

Quantization is the process of representing a model’s weights (and/or activations) using a lower-precision numeric format – e.g. 8-bit integers (int8) or 4-bit integers (int4) – instead of the original 32-bit or 16-bit floating point (fp32/fp16) format the model was trained in. This shrinks model size and memory bandwidth requirements roughly in proportion to the bit-width reduction (e.g. int8 is $\sim 4\times$ smaller than fp32) and speeds up inference on hardware with fast integer arithmetic, since integer matrix multiplications are cheaper than floating point ones and moving fewer bytes reduces memory-bandwidth bottlenecks.

The accuracy trade-off. Lower precision means fewer distinct representable values over the same range, which introduces *quantization error* – the difference between the true fp32 value and its rounded low-precision approximation. As the bit-width drops:

- **int8** typically loses very little accuracy (often $<1\%$ degradation) because 256 levels are usually enough to represent weight distributions faithfully, especially with per-channel scaling.
- **int4** has only 16 levels, so the rounding error is much larger. It can noticeably hurt accuracy on complex tasks unless mitigated with more advanced calibration techniques (e.g. GPTQ, AWQ, per-group quantization) that decide scales more carefully than a single global scale.

In short: fewer bits $\rightarrow$ smaller / faster model, but larger rounding error $\rightarrow$ higher risk of accuracy loss, and the severity depends heavily on the quantization scheme used, not just the bit-width.

Example: symmetric int8 quantization by hand. The most common scheme is *symmetric linear quantization*. Given a tensor of fp32 values, we compute a single scale factor s from the maximum absolute value in the tensor, then map every value to the nearest integer in $[-127, 127]$:

$$s = \frac{\max(|x|)}{127}, \quad x_q = \text{round}\left(\frac{x}{s}\right), \quad x_{\text{dequant}} = x_q \cdot s$$

Consider a small weight vector $x = [0.25, -0.75, 1.00, -0.30, 0.05]$. The maximum absolute value is 1.00, so:

$$s = \frac{1.00}{127} \approx 0.007874$$

Applying the formula to each element gives Table 1.

Table 1: Symmetric int8 quantization of an example weight vector ($s \approx 0.007874$)

Original (x)	x/s	Quantized int8 (x_q)	Dequantized ($x_q \cdot s$)	Abs. error
0.25	31.75	32	0.2520	0.0020
-0.75	-95.25	-95	-0.7480	0.0020
1.00	127.00	127	1.0000	0.0000
-0.30	-38.10	-38	-0.2992	0.0008
0.05	6.35	6	0.0472	0.0028

Each value now needs only 1 byte instead of 4 bytes (fp32), a $4\times$ memory reduction, at the cost of a small rounding error (at most ~ 0.003 in this example, i.e. well under 1% of the value range) – a good illustration of the accuracy/size trade-off in general: the error is small here because the vector’s dynamic range is modest and int8 has enough levels to cover it, but the same scheme applied with only int4 (16 levels instead of 256) would produce roughly $8\times$ coarser steps and proportionally larger errors.

Question 4: A Real Inference Serving Framework

vLLM is a widely used open-source LLM inference and serving engine. Its headline optimization is **PagedAttention**, which manages the KV cache the way an operating system manages virtual memory:

- Instead of allocating one large, contiguous block of memory per request for its entire (worst-case) sequence length – which wastes a lot of memory to internal/external fragmentation, since most sequences end up shorter than the reserved maximum – vLLM splits the KV cache into small fixed-size “pages” (blocks).
- Each sequence’s KV cache is stored as a list of these non-contiguous pages, referenced through a per-sequence page table, similar to how an OS maps virtual pages to physical memory frames.
- This allows memory to be allocated on demand, shared between sequences that have common prefixes (e.g. the same system prompt), and reclaimed immediately when a sequence finishes, dramatically reducing memory waste.
- Less wasted memory means more sequences can be batched together concurrently, which directly translates into higher serving throughput (vLLM’s original paper reports large throughput gains over naive HF serving implementations at the time).